

FACULDADE COTEMIG
Fundamentos Teóricos da Computação

Trabalho Final

Implementação de Máquinas Abstratas: AFD, Autômato de Pilha e Máquina de Turing

Integrantes:

Bernardo Santos Magalhães — mat. 72400188

João Victor Amaro Araújo — mat. 72400110

Lucas Filipe França Nunes — mat. 72401362

Professor: Júlio César da Silva

Belo Horizonte - MG, 2026

Sumário

1	Introdução	2
2	Fundamentação Teórica	2
2.1	Autômato Finito Determinístico (AFD)	2
2.2	Autômato de Pilha (AP) com reconhecimento por pilha vazia	3
2.3	Máquina de Turing (MT)	3
3	Descrição da Implementação	4
3.1	Autômato Finito Determinístico (AFD)	4
3.2	Autômato de Pilha (AP)	6
3.3	Máquina de Turing (MT)	7
4	Resultados e Testes	9
4.1	AFD — L_1	9
4.2	Autômato de Pilha — L_2 e L_3	10
4.3	Máquina de Turing — L_4	10
5	Análise Comparativa	11
6	Conclusão	12
	Referências	13

1 Introdução

Este relatório documenta o trabalho final da disciplina de Fundamentos Teóricos da Computação, cujo objetivo é compreender, modelar e implementar três máquinas abstratas fundamentais da Teoria da Computação: o Autômato Finito Determinístico (AFD), o Autômato de Pilha (AP) com reconhecimento por pilha vazia e a Máquina de Turing (MT). Cada máquina corresponde a uma classe de linguagens distinta na Hierarquia de Chomsky (MENEZES, 1998) e a um modelo de poder computacional crescente.

As implementações foram desenvolvidas na linguagem C# (.NET 6 ou superior), sem o uso de bibliotecas externas de autômatos, de modo que o código seja uma tradução direta das definições formais estudadas em aula. Para cada máquina, foi escolhida uma linguagem-alvo representativa: L_1 , as palavras sobre $\{a, b\}$ que terminam com “ab” (regular); $L_2 = \{a^n b^n \mid n \geq 1\}$ (livre de contexto); e $L_4 = \{a^n b^n c^n \mid n \geq 1\}$ (sensível ao contexto, reconhecida por MT). Além de reconhecer as linguagens, foram implementados os desafios obrigatórios de cada parte.

2 Fundamentação Teórica

2.1 Autômato Finito Determinístico (AFD)

Um AFD é a quintupla $M = (Q, \Sigma, \delta, q_0, F)$, em que Q é um conjunto finito de estados; Σ é o alfabeto de entrada; $\delta : Q \times \Sigma \rightarrow Q$ é a função de transição; $q_0 \in Q$ é o estado inicial; e $F \subseteq Q$ é o conjunto de estados finais (de aceitação). O determinismo decorre de δ ser uma função: para cada par (estado, símbolo) existe exatamente um estado de destino.

O processamento de uma palavra inteira é descrito pela **função de transição estendida** $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$, definida recursivamente por $\hat{\delta}(q, \lambda) = q$ e $\hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a)$, em que λ denota a palavra vazia. A linguagem aceita por M é $L(M) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F\}$.

A linguagem-alvo é $L_1 = \{w \in \{a, b\}^* \mid w \text{ termina com } ab\}$. O AFD construído é $M = (\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_2\})$, com δ dada pela Tabela 1. Os estados resumem o sufixo já lido: q_0 (sem sufixo útil), q_1 (o último símbolo foi a) e q_2 (acabou de ler ab , único estado de aceitação).

Tabela 1: Função de transição δ do AFD de L_1 .

Estado	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
$*q_2$	q_1	q_0

Fonte: Elaborada pelos autores (2026).

Cálculo manual (via $\hat{\delta}$). Para a cadeia bab (esperado: ACEITA): $\hat{\delta}(q_0, b) = q_0$; $\hat{\delta}(q_0, ba) = \delta(q_0, a) = q_1$; $\hat{\delta}(q_0, bab) = \delta(q_1, b) = q_2 \in F \Rightarrow$ **ACEITA**. Para a cadeia ba (esperado: RE-

JEITA): $\hat{\delta}(q_0, ba) = \delta(q_0, a) = q_1 \notin F \Rightarrow \mathbf{REJEITA}$.

2.2 Autômato de Pilha (AP) com reconhecimento por pilha vazia

Um AP é a sêtucla $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, \emptyset)$, em que Γ é o alfabeto da pilha, $Z_0 \in \Gamma$ é o símbolo inicial da pilha, e $\delta : Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma^*)$. Neste modelo, a aceitação ocorre **exclusivamente por pilha vazia**: uma palavra w é aceita se existe uma computação que, ao consumir w , esvazia completamente a pilha (o conjunto de estados de aceitação é vazio).

Cada transição é escrita na forma $\delta(e, a, b) = [e', z]$, que se lê “no estado e , lendo a , desempilhando b , vai para e' empilhando z ”. As computações são descritas por **configurações instantâneas** $[e, w, \gamma]$ (estado, entrada restante e pilha, com o topo à esquerda) e pela relação de derivação $\vdash$, conforme $[e, ay, b\gamma] \vdash [e', y, z\gamma]$ sempre que $\delta(e, a, b) = [e', z]$. Observa-se que os slides apresentam o autômato de pilha determinístico como uma sêxtupla com estados finais; o enunciado, por sua vez, exige a sêtucla com Z_0 e aceitação por pilha vazia. Ambos os modos de aceitação são equivalentes em poder de reconhecimento (discutido nas questões reflexivas).

A linguagem-alvo é $L_2 = \{a^n b^n \mid n \geq 1\}$. A estratégia empilha um marcador X para cada a lido e desempilha um X para cada b ; ao final, um λ -movimento desempilha Z_0 , esvaziando a pilha. A computação de $aabb$ (aceita) é:

$$[q_0, aabb, Z] \vdash [q_0, abb, XZ] \vdash [q_0, bb, XXZ] \vdash [q_1, b, XZ] \vdash [q_1, \lambda, Z] \vdash [q_1, \lambda, \lambda].$$

Já aab (rejeitada) trava em $[q_1, \lambda, XZ]$: a entrada acaba, mas resta um X na pilha, que portanto não se esvazia. O desafio acrescenta um segundo AP para os palíndromos $L_3 = \{w \in \{a, b\}^* \mid w = w^R, |w| \geq 1\}$; diferentemente de L_2 , L_3 exige um autômato de pilha **não determinístico**, pois é preciso “adivinhar” o meio da palavra.

2.3 Máquina de Turing (MT)

Conforme Sipser (2010), uma MT é a sêtucla $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, com fita infinita, alfabeto da fita $\Gamma \supseteq \Sigma$ contendo o símbolo branco ($\sqcup$), e $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$. A MT lê e *escreve* na fita e move o cabeçote para a esquerda (L) ou direita (R).

A linguagem-alvo é $L_4 = \{a^n b^n c^n \mid n \geq 1\}$, que **não** é livre de contexto (uma pilha LIFO compara duas grandezas, mas não três ao mesmo tempo); ela é, contudo, sensível ao contexto e, portanto, reconhecível por uma MT. A estratégia adotada é de **marcação**: a cada passada, a máquina marca um a (reescrevendo-o como X), o primeiro b (como Y) e o primeiro c (como Z). Como cada passada consome exatamente um símbolo de cada bloco, as três contagens só se esgotam simultaneamente quando $n_a = n_b = n_c$. Esgotados os a , uma varredura final confirma o padrão $X^*Y^*Z^*$ seguido do branco e a cadeia é aceita. O autômato construído emprega oito estados (q_0 a q_5 , além de q_{accept} e q_{reject}), detalhados na Seção 3.3.

Cálculo manual (configurações instantâneas). Escrevendo cada configuração como (estado :

fita), com o símbolo sob o cabeçote entre colchetes, a computação de abc (esperado: ACEITA) é:

$q_0 : [a]bc \vdash q_1 : X[b]c \vdash q_2 : XY[c] \vdash q_3 : X[Y]Z \vdash q_3 : [X]YZ \vdash q_0 : X[Y]Z \vdash q_4 : XY[Z] \vdash q_5 : XYZ[\sqcup] \vdash q_{acc}$

Já ab (esperado: REJEITA): $q_0 : [a]b \vdash q_1 : X[b] \vdash q_2 : XY[\sqcup]$; em q_2 , lendo o branco, não há transição definida (falta o c correspondente ao b já marcado), de modo que a máquina para em $q_{reject} \Rightarrow \text{REJEITA}$.

3 Descrição da Implementação

3.1 Autômato Finito Determinístico (AFD)

O simulador foi implementado em C# como aplicação de console, sem bibliotecas externas de autômatos. A classe AFD traduz diretamente a quintupla formal: cada elemento matemático corresponde a um campo, conforme a Tabela 2.

Tabela 2: Correspondência entre a quintupla formal e as estruturas de dados.

Elemento	Campo em C#	Estrutura
Q	Estados	HashSet<string>
Σ	Alfabeto	HashSet<char>
δ	Transicao	Dictionary<(string, char), string>
q_0	EstadoInicial	string
F	EstadosAceitacao	HashSet<string>

Fonte: Elaborada pelos autores (2026).

Os conjuntos Q , Σ e F usam HashSet por representarem conjuntos (sem repetição e com teste de pertinência eficiente). A função δ é um Dictionary cuja chave é o par (estado, símbolo) e cujo valor é o estado de destino, espelhando $\delta : Q \times \Sigma \rightarrow Q$.

O método Processar implementa, de forma iterativa, a função estendida $\hat{\delta}$: parte de q_0 e, para cada símbolo, consulta δ para obter o próximo estado, registrando o rastro de estados percorridos; ao final, aceita se o estado corrente pertence a F . O método público bool Aceitar(string) apenas devolve esse veredito. O método ExibirDiagrama imprime a tabela de transições no console.

Como δ não é total, uma transição indefinida ou um símbolo fora de Σ leva a um **estado-poço implícito**: a cadeia é imediatamente rejeitada, preservando o determinismo e evitando exceções. O desafio obrigatório é atendido pela classe CarregadorJson, que reconstrói qualquer AFD a partir de um arquivo afd.json usando a biblioteca nativa System.Text.Json, validando a consistência da quintupla resultante.

Questões reflexivas — AFD

1. Por que L_1 é regular? Expressão regular equivalente.

L_1 é regular porque pode ser reconhecida por um AFD, isto é, por uma máquina de memória finita: para decidir se uma palavra termina com “ab”, basta lembrar um resumo finito do que já foi lido — se o sufixo atual é vazio, “a” ou “ab” —, o que assume apenas três possibilidades (os três estados do autômato). Pelo Teorema de Kleene, toda linguagem reconhecida por um autômato finito possui uma expressão regular equivalente; como exibimos um AFD para L_1 , ela é necessariamente regular. Pode-se ainda argumentar pela relação de Myhill-Nerode, que possui índice finito (três classes de equivalência). A expressão regular equivalente é $(a|b)^*ab$, ou, em notação de alfabeto, Σ^*ab com $\Sigma = \{a, b\}$. Outra forma de enxergar a regularidade é notar que L_1 é a concatenação da linguagem regular Σ^* com a palavra fixa ab, e a classe das linguagens regulares é fechada sob concatenação, logo L_1 é necessariamente regular. Como o autômato possui apenas três estados e nenhum deles precisa “contar” uma quantidade ilimitada de símbolos, a memória finita do AFD é suficiente, o que confirma que L_1 não demanda um modelo computacional mais poderoso.

2. Descreva formalmente a quintupla e justifique cada estado.

O AFD é $M = (\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_2\})$, com δ dada pela Tabela 1. Cada estado carrega um significado ligado ao progresso na formação do sufixo “ab”: q_0 representa “ainda não há um a iniciando o sufixo”, sendo também o estado de recomeço após um b ; q_1 significa “o último símbolo lido foi a ”, um candidato a formar “ab”; e q_2 , único de aceitação, significa “acabou de ler ab”. Como $F = \{q_2\}$, a palavra só é aceita se a leitura terminar exatamente nesse estado, capturando a condição de terminar com “ab”. Observa-se ainda que δ é total sobre $\{a, b\}$: cada um dos três estados possui transição definida tanto para a quanto para b , o que assegura o determinismo e evita travamentos durante a leitura. Não foi necessário um estado-poço explícito, pois toda combinação (estado, símbolo) do alfabeto conduz a um estado válido; a rejeição decorre unicamente de a computação não encerrar em q_2 .

3. E se δ não fosse total? Como tratar entradas inválidas?

Na definição teórica, δ é total: há destino para todo par (estado, símbolo). Se δ não for total, alguns pares ficam sem transição e o autômato pode “travar”. A solução padrão é introduzir um **estado-poço**: um estado não-final do qual não se sai, para onde se direcionam as transições ausentes, tornando δ total novamente. Na implementação, adotou-se δ parcial com estado-poço implícito: se o par não existe na tabela, ou se o símbolo está fora de Σ (por exemplo, um “c”), a cadeia é imediatamente rejeitada com uma observação. Isso preserva o determinismo, evita exceções e mantém o comportamento previsível. Convém distinguir dois casos: (i) pares (estado, símbolo) do próprio alfabeto sem transição, que não ocorrem neste AFD, pois δ é total sobre $\{a, b\}$; e (ii) símbolos fora de Σ , como c, que jamais teriam destino. Em ambos, a rejeição imediata equivale, na prática, a um estado-poço absorvente e não-final, preservando a totalidade lógica de δ exigida pela definição formal sem a necessidade de codificá-lo explicitamente no

programa.

3.2 Autômato de Pilha (AP)

A classe `AutomatoPilha` traduz a s etupla formal: os campos `Estados` (Q), `Alfabeto` (Σ), `AlfabetoPilha` (Γ), `EstadoInicial` (q_0) e `SimboloInicialPilha` (Z_0). N o h a conjunto de estados de aceita  o, pois a aceita  o   por pilha vazia. A pilha   implementada com `Stack<char>`, conforme exigido (n o se usou `LinkedList` nem vetores). A palavra vazia λ   representada pelo caractere `'\0'`.

A fun  o δ   um `Dictionary` cuja chave   a tripla (`estado`, `entrada`, `topo`) e cujo valor   uma *lista* de pares (`novoEstado`, `empilha`). O uso de uma lista permite representar o **n o determinismo** (v rios destinos para a mesma configura  o), necess rio para L_3 . Um s mbolo de entrada `'\0'` indica λ -movimento (n o consome entrada); um s mbolo de topo `'\0'` indica que o topo   ignorado (n o h a desempilhamento).

Como o aut mato pode ser n o determin stico, a simula  o faz uma **busca em profundidade** sobre as configura  es (`estado`, `posi  o na entrada`, `pilha`), com um conjunto de configura  es visitadas e um limite de passos para evitar la os infinitos de λ -movimentos. A cada ramo, a `Stack<char>`   clonada. Uma cadeia   aceita ao alcan ar uma configura  o com a entrada totalmente consumida e a pilha vazia. O m todo de simula  o exibe, passo a passo, a **configura  o instant nea** (`estado`, `entrada restante` e `pilha`), no estilo da rela  o $\vdash$. Foram constru dos dois aut matos: o de L_2 (determin stico) e o de L_3 (n o determin stico, lendo as cadeias de entradas `palindromos.txt`).

Quest es reflexivas — Aut mato de Pilha

1. Por que L_2 n o pode ser reconhecida por um AFD? (Lema do Bombeamento.)

Suponha, por absurdo, que $L_2 = \{a^n b^n \mid n \geq 1\}$ seja regular. Ent o existe uma constante de bombeamento p tal que toda palavra $w \in L_2$ com $|w| \geq p$ pode ser escrita como $w = xyz$, com $|xy| \leq p$, $|y| \geq 1$ e $xy^i z \in L_2$ para todo $i \geq 0$. Tome $w = a^p b^p \in L_2$. Como $|xy| \leq p$, o trecho xy   formado apenas por s mbolos a , logo $y = a^k$ com $k \geq 1$. Ao bombear com $i = 2$, obt m-se $xy^2 z = a^{p+k} b^p$, que tem mais a s do que b s e, portanto, n o pertence a L_2 — contradi  o. Conclu mos que L_2 n o   regular. Intuitivamente, um AFD possui mem ria finita (seus estados) e n o consegue “contar” um n mero arbitr rio de a s para depois compar -lo ao n mero de b s;   exatamente essa contagem ilimitada que a pilha do AP fornece.

2. A aceita  o por pilha vazia e por estado final s o equivalentes?

Sim: os dois modos reconhecem exatamente a mesma classe (as linguagens livres de contexto), de modo que nenhum   mais poderoso que o outro. A equival ncia   mostrada por constru  o. Dado um AP que aceita por *pilha vazia*, constr i-se um equivalente que aceita por *estado final*: introduz-se um novo s mbolo de fundo X_0 (empilhado abaixo de Z_0) e um novo estado final alcan ado por um λ -movimento quando X_0 volta a aparecer no topo, ou seja, quando a pilha

“original” se esvaziou. Reciprocamente, dado um AP que aceita por estado final, constrói-se um que aceita por pilha vazia: ao entrar em um estado final, o autômato passa a um estado especial que, por λ -movimentos, desempilha todo o conteúdo restante, esvaziando a pilha. Neste trabalho, adotou-se a aceitação por pilha vazia (com $F = \emptyset$), como exige o enunciado, ao passo que os slides utilizam estados finais; a equivalência garante que a escolha não altera a linguagem reconhecida.

3. Qual o papel do símbolo Z_0 na implementação?

O símbolo Z_0 marca o **fundo da pilha**. Ele é empilhado na inicialização e permite ao autômato distinguir, durante a computação, uma pilha que ainda contém marcadores de uma pilha “logicamente vazia”. No AP de L_2 , os marcadores X ficam acima de Z_0 ; somente depois de todos os X terem sido desempilhados (isto é, quando o número de bs igualou o de as) é que Z_0 reaparece no topo, habilitando o λ -movimento final $\delta(q_1, \lambda, Z_0) = [q_1, \lambda]$, que esvazia a pilha e provoca a aceitação. Sem Z_0 , não haveria um gatilho seguro para esse passo final, e o autômato correria o risco de esvaziar a pilha prematuramente, aceitando palavras indevidas. Assim, Z_0 funciona como sentinela de fundo e como condição de disparo da aceitação por pilha vazia.

3.3 Máquina de Turing (MT)

A classe `MaquinaTuring` traduz a sétupla formal em campos nomeados: Estados (Q , `HashSet<string>`), Alfabeto (Σ , `HashSet<char>`), AlfabetoFita (Γ , `HashSet<char>`), EstadoInicial (q_0), EstadoAceitacao (q_{accept}) e EstadoRejeicao (q_{reject}). A fita é uma estrutura **dinâmica** `Dictionary<int, char>`, em que a chave é a posição (um inteiro que pode crescer para a direita ou para a esquerda) e o valor é o símbolo gravado; posições nunca escritas valem o branco, representado por ‘_’. Essa escolha modela a fita potencialmente infinita sem reservar memória a priori, ao contrário de um vetor de tamanho fixo.

A função de transição δ é um `Dictionary<(string estado, char simbolo), (string novoEstado, char novoSimbolo, char direcao)>`, exatamente o tipo exigido no enunciado: a chave é o par (estado, símbolo lido) e o valor é a tripla (novo estado, símbolo a escrever, direção), com a direção ‘L’ (esquerda) ou ‘R’ (direita). Como δ é **parcial**, um par sem transição definida corresponde, por convenção, à parada em q_{reject} , espelhando o estado de rejeição implícito da MT. O construtor executa `ValidarConsistencia()`, que confere as restrições da definição formal: $\sqcup \notin \Sigma$, $\sqcup \in \Gamma$, $\Sigma \subseteq \Gamma$, os três estados especiais pertencendo a Q , $q_{\text{accept}} \neq q_{\text{reject}}$ e toda transição usando apenas estados de Q , símbolos de Γ e direção válida.

O método `Executar` simula a máquina a partir de q_0 , com o cabeçote na posição 0. A cada passo ele lê o símbolo sob o cabeçote, consulta δ , grava o novo símbolo, move o cabeçote (+1 para R, -1 para L) e troca de estado, registrando a configuração instantânea. A execução para ao alcançar q_{accept} (aceita) ou q_{reject} (rejeita). Um **contador de passos** e um **limite configurável** (`LimitePassos`, padrão 10000, passado no construtor) interrompem laços infinitos: ao atingir o limite, o resultado é marcado como “não parou”. A configuração

instantânea exibida a cada passo mostra o **estado atual**, o **conteúdo completo da fita** com delimitadores [] ao redor do símbolo sob o cabeçote e a **posição do cabeçote**, atendendo à exigência (c) do enunciado. O método `ExibirDefinicao` imprime a 7-tupla e a tabela de δ na notação dos slides, $\delta(q, a) = [q', b, d]$.

Estratégia e estados de L_4 . Foram necessários **oito estados**, com a marcação $X/Y/Z$ descrita na fundamentação: q_0 inicia cada passada sobre o a mais à esquerda ainda não marcado; q_1 procura o primeiro b ; q_2 procura o primeiro c ; q_3 retorna à esquerda até o último X e reinicia a passada; q_4 e q_5 realizam a varredura final sobre Y^* e Z^* ; e $q_{\text{accept}}/q_{\text{reject}}$ são os estados de parada. Cada transição está comentada no código com seu significado semântico.

Desafio — MT transdutora. Uma segunda MT *computa* (não apenas reconhece) $f(n) = n + 1$ em unário. Com apenas dois estados úteis (q_0 e q_{accept}), ela percorre os n símbolos '1' para a direita até o primeiro branco e, nele, escreve mais um '1', deixando a fita com $n + 1$ ocorrências; o caso $n = 0$ (entrada vazia) também é tratado, produzindo um único '1' ($f(0) = 1$). Diferentemente da MT de L_4 , que apenas decide pertinência, esta usa a fita como **saída**, ilustrando o papel da MT como modelo de computação de funções.

As cadeias são lidas dos arquivos `entradas_mt.txt` e `entradas_unario.txt`. Uma função auxiliar deriva da própria estrutura da cadeia uma justificativa legível do veredito (por exemplo, “ $n = 2$ ” ou “quantidades distintas ($a=2, b=2, c=1$)”); ressalte-se que quem *decide* de fato é a Máquina de Turing — a justificativa é apenas explicativa, exibida ao lado do rastro.

Questões reflexivas — Máquina de Turing

1. Por que L_4 não pode ser reconhecida por um AP? Qual propriedade da MT permite reconhecê-la?

A linguagem $L_4 = \{a^n b^n c^n \mid n \geq 1\}$ não é livre de contexto, logo nenhum autômato de pilha a reconhece. Isso pode ser mostrado pelo Lema do Bombeamento para linguagens livres de contexto (HOPCROFT; MOTWANI; ULLMAN, 2002): supondo L_4 livre de contexto, haveria uma constante p tal que $z = a^p b^p c^p$ se escreve como $z = uvwxy$, com $|vwx| \leq p$, $|vx| \geq 1$ e $uv^i wx^i y \in L_4$ para todo i . Como $|vwx| \leq p$, esse trecho não alcança os três blocos ao mesmo tempo — abrange, no máximo, dois símbolos distintos. Ao bombear (por exemplo, $i = 2$), pelo menos um dos três blocos fica sem ser incrementado, quebrando a igualdade $n_a = n_b = n_c$; logo, a palavra resultante não está em L_4 — contradição. Intuitivamente, a pilha é LIFO e só permite comparar *duas* grandezas: ao casar os a com os b , os marcadores são desempilhados e a contagem se perde, restando memória insuficiente para comparar também os c . A propriedade da MT que viabiliza o reconhecimento é a fita de **leitura e escrita** com cabeçote bidirecional: a máquina pode marcar, visitar e reescrever símbolos quantas vezes precisar, dispondo de memória de trabalho ilimitada e com acesso a qualquer posição, o que lhe permite acoplar as três contagens, marcando um a , um b e um c por passada.

2. Quantos estados foram necessários para a MT de L_4 ? Estratégia de marcação.

A MT utiliza **oito estados**: seis de trabalho (q_0 a q_5) e os dois de parada, q_{accept} e q_{reject} . A

estratégia é de **marcação**: a cada passada, a máquina troca um a pelo marcador X (estado q_0), caminha para a direita até o primeiro b , trocando-o por Y (q_1), e segue até o primeiro c , trocando-o por Z (q_2). Em seguida, q_3 retorna à esquerda até o último X gravado e reinicia a passada sobre o próximo a . Como cada passada consome **exatamente um** símbolo de cada bloco, as três contagens só se esgotam juntas quando $n_a = n_b = n_c$. Quando não restam a (o estado q_0 lê Y), inicia-se a verificação final: q_4 percorre os Y e q_5 percorre os Z ; se ao fim encontra-se o branco, a fita tem a forma $X^*Y^*Z^*$ com blocos de igual tamanho e a cadeia é aceita. Qualquer desvio desse padrão (símbolo inesperado ou fim prematuro) cai em transição indefinida e leva a q_{reject} .

3. Um computador moderno é mais poderoso que uma Máquina de Turing? (Tese de Church-Turing.)

Não. Pela **Tese de Church-Turing** (SIPSER, 2010), tudo o que pode ser calculado por um procedimento efetivo (um algoritmo) pode ser calculado por uma Máquina de Turing; nenhum modelo fisicamente realizável de computação supera o poder da MT. Um computador moderno é incomparavelmente mais *rápido* e conveniente, mas não computa nada que uma MT não compute: ele permanece incapaz de resolver problemas indecidíveis, como o Problema da Parada. Na verdade, em sentido estrito, o computador real é até *menos* poderoso, pois possui memória **finita** — equivalendo formalmente a um autômato finito muito grande —, ao passo que a MT pressupõe fita ilimitada. Por isso, a MT é usada como **limite superior** ideal da computabilidade algorítmica: é a referência teórica do que é “computável”. Em resumo, mais velocidade e mais memória não ampliam a *classe* de problemas solucionáveis; apenas tornam viável, na prática, o que já era computável em teoria.

4 Resultados e Testes

4.1 AFD — L_1

A Tabela 3 apresenta os sete casos obrigatórios e a saída produzida pelo programa.

Tabela 3: Resultados dos casos de teste do AFD.

Cadeia	Rastro de estados	Obtido	Enunciado
ab	$q_0q_1q_2$	ACEITA	ACEITA
aab	$q_0q_1q_1q_2$	ACEITA	REJEITA*
bab	$q_0q_0q_1q_2$	ACEITA	ACEITA
ababab	$q_0q_1q_2q_1q_2q_1q_2$	ACEITA	ACEITA
ba	$q_0q_0q_1$	REJEITA	REJEITA
λ	q_0	REJEITA	REJEITA
b	q_0q_0	REJEITA	REJEITA

Fonte: Saída do programa desenvolvido pelos autores (2026).

***Observação sobre o caso aab.** A palavra aab é formada por $a-a-b$; seus dois últimos símbo-

los são “ab”, logo ela *termina com ab* e, pela definição de L_1 e pela expressão regular equivalente $(a|b)^*ab$, o resultado correto é **ACEITA**. A tabela do enunciado registra aab = REJEITA com a justificativa “não termina com ab”, o que é factualmente incorreto. Optou-se por manter a fidelidade à definição formal, aceitando aab, e registra-se aqui a divergência por transparência.

4.2 Autômato de Pilha — L_2 e L_3

A Tabela 4 apresenta os casos obrigatórios de L_2 e os casos do desafio (L_3 , palíndromos), com a saída produzida pelo programa.

Tabela 4: Resultados dos casos de teste do Autômato de Pilha.

Linguagem	Cadeia	Obtido	Esperado
L_2	ab	ACEITA	ACEITA
L_2	aabb	ACEITA	ACEITA
L_2	aaabbb	ACEITA	ACEITA
L_2	aab	REJEITA	REJEITA
L_2	abb	REJEITA	REJEITA
L_2	ba	REJEITA	REJEITA
L_2	λ	REJEITA	REJEITA
L_2	abab	REJEITA	REJEITA
L_3	a	ACEITA	ACEITA
L_3	aba	ACEITA	ACEITA
L_3	abba	ACEITA	ACEITA
L_3	ab	REJEITA	REJEITA
L_3	aab	REJEITA	REJEITA

Fonte: Saída do programa desenvolvido pelos autores (2026).

4.3 Máquina de Turing — L_4

A Tabela 5 apresenta os sete casos obrigatórios de L_4 e a saída produzida pelo programa, incluindo o estado de parada e o número de passos.

Tabela 5: Resultados dos casos de teste da MT para $L_4 = a^n b^n c^n$.

Cadeia	Parou em	Passos	Obtido	Esperado
abc	q_{accept}	8	ACEITA	ACEITA
aabbcc	q_{accept}	23	ACEITA	ACEITA
aaabbbccc	q_{accept}	46	ACEITA	ACEITA
aabbcc	q_{reject}	13	REJEITA	REJEITA
ab	q_{reject}	2	REJEITA	REJEITA
abc abc	q_{reject}	7	REJEITA	REJEITA
λ	q_{reject}	0	REJEITA	REJEITA

Fonte: Saída do programa desenvolvido pelos autores (2026).

Todos os sete casos produziram o resultado esperado pelo enunciado, diferentemente da Parte 1, em que o caso aab continha um equívoco na tabela do enunciado, aqui não há divergências.

Observação sobre o caso abc abc. Conforme lido do arquivo, a linha contém um *espaço* entre os dois blocos. Esse espaço não pertence ao alfabeto $\Sigma = \{a, b, c\}$; ao alcançá-lo, a MT não encontra transição definida e para em q_{reject} . Ainda que o espaço fosse removido, a cadeia abcababc também seria rejeitada: após marcar o primeiro bloco, o segundo *a* surge onde a varredura final só admite *Y*, *Z* ou branco, levando igualmente a q_{reject} . Em ambas as leituras o veredito REJEITA é robusto.

Desafio — MT transdutora $f(n) = n + 1$. A Tabela 6 mostra que, para cada entrada unária, a fita final contém exatamente $n + 1$ ocorrências de '1', conforme exigido.

Tabela 6: Resultados da MT que computa $f(n) = n + 1$ em unário.

Entrada	n	Fita de saída	Esperado
1	1	11	11
111	3	1111	1111
11111	5	111111	111111

Fonte: Saída do programa desenvolvido pelos autores (2026).

5 Análise Comparativa

As três máquinas formam uma hierarquia de poder computacional crescente, correspondente à Hierarquia de Chomsky (MENEZES, 1998; LEWIS; PAPADIMITRIOU, 2000). O AFD reconhece exatamente as **linguagens regulares**, dispondo apenas de memória finita (seus estados). Por isso, reconhece L_1 , mas é incapaz de reconhecer $L_2 = a^n b^n$, pois isso exigiria “contar” uma quantidade ilimitada de *as*, o que o Lema do Bombeamento para linguagens regulares demonstra ser impossível com memória finita.

O Autômato de Pilha acrescenta uma **pilha** como memória auxiliar, reconhecendo as **linguagens livres de contexto**. Essa pilha permite contar e comparar duas grandezas (os *as* e os *bs* de L_2), mas sua natureza LIFO impede comparar três grandezas simultaneamente. Assim, o AP reconhece L_2 , mas não $L_4 = a^n b^n c^n$.

A Máquina de Turing, com sua **fita infinita** de leitura e escrita, é o modelo mais poderoso, reconhecendo L_4 e, de modo geral, todas as linguagens recursivamente enumeráveis; pela Tese de Church-Turing, captura a noção intuitiva de algoritmo. A MT pode, ainda, *computar funções* (e não apenas decidir pertinência), como ilustra o desafio $f(n) = n + 1$. Em síntese: linguagens regulares $\subsetneq$ livres de contexto $\subsetneq$ decidíveis por MT.

6 Conclusão

O desenvolvimento deste trabalho permitiu traduzir definições formais da Teoria da Computação em implementações funcionais, evidenciando a correspondência direta entre os formalismos matemáticos e as estruturas de dados. A construção do AFD reforçou o conceito de memória finita e da função de transição estendida; o Autômato de Pilha mostrou como uma única pilha (com o sentinela Z_0) acrescenta a capacidade de contar e comparar duas grandezas; e a Máquina de Turing, por meio da fita de leitura e escrita e da estratégia de marcação, evidenciou o salto de poder que permite reconhecer L_4 e até computar funções, como $f(n) = n + 1$. A comparação entre as três máquinas tornou concreta a Hierarquia de Chomsky e os limites de cada modelo. Conclui-se que a escolha do modelo computacional adequado depende diretamente da estrutura da linguagem a ser reconhecida e que, embora a Máquina de Turing seja o modelo mais poderoso — limite da computabilidade algorítmica segundo a Tese de Church-Turing (SIPSER, 2010) —, é também o mais custoso de projetar e depurar.

Referências

HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. *Introdução à teoria de autômatos, linguagens e computação*. 2. ed. Rio de Janeiro: Elsevier, 2002.

LEWIS, Harry R.; PAPADIMITRIOU, Christos H. *Elementos de teoria da computação*. 2. ed. Porto Alegre: Bookman, 2000.

MENEZES, Paulo Blauth. *Linguagens formais e autômatos*. Porto Alegre: Sagra Luzzatto, 2010.

SIPSER, Michael. *Introdução à teoria da computação*. 2. ed. São Paulo: Cengage Learning, 2010.